ucc-next

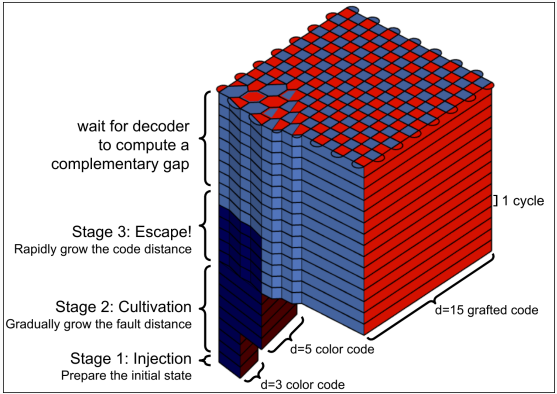
A Multi-Level Compiler & Simulation Enginefor the Early Fault Tolerance Era

March 12, 2026

The Simulation Bottleneck & Our Approach

Motivating Example: Magic State Cultivation (MSC)

The Goal: Evaluate Magic State Cultivation protocols over 10^{12} shots.



The Challenge

- Cultivation includes non-Clifford T gates, so Stim does not apply.
- End-to-end 463-qubit cultivation is intractable with existing simulators:
  - **Statevector (Qiskit):** $\mathcal{O}(2^N)$
  - **Stabilizer rank (tsim):** $\mathcal{O}(2^{0.4t})$
  - **Generalized stabilizers (SOFT):** heavy $\mathcal{O}(n^2)$ per-shot overhead

The Approach: Making Simulation Feasible

To simulate early fault-tolerant circuits at scale, we must avoid the exponential walls of standard exact simulators.

The Core Insight:

- We break the exponential bottlenecks by cleanly decoupling deterministic routing from probabilistic non-Clifford entanglement.
- By tracking these separately, the simulation scales exponentially *only* with the **Peak Active Rank** ($k_{\max}$), bypassing the physical qubit (N) and T-count (t) walls entirely.
- But beyond SOFT, we do the $k_{\max}$ scaling *without* incurring the $\mathcal{O}(n^2)$ per-shot overhead.

The “New” UCC: Driven by the Math

This decoupled mathematical approach demands a fundamentally different software architecture. It suggests a split design to handle the deterministic and probabilistic regimes separately.

Previous UCC: An open-source routing and bridging compiler.

New UCC: A **Multi-Level Compiler** and **Virtual Machine (VM)** built explicitly for the Early FTQC Era.

- **The Compiler (Offline):** Deterministically resolves and absorbs the Clifford routing overhead.
- **The VM (Online):** Executes the isolated probabilistic non-Clifford operations at runtime.

The Core Theory: The Factored State

Circuit Composition & The Heisenberg Mapping

We partition all quantum operations into two sets:

1. **Deterministic:** Clifford Unitaries ($CNOT, H, \ldots$).
2. **Active:** Non-Clifford rotations (T), Pauli Measurements, Pauli Noise, Conditional Paulis.

This covers a universal gateset, most common noise models and feed-forward mid-circuit measurement.

It doesn't cover conditional non-clifford gates.

The Factored State Equation

We define the exact physical quantum state at time t via a strict factorization:

$$|\psi^{(t)}\rangle = \gamma^{(t)} U_C^{(t)} \tilde{P}^{(t)} \left(|\phi^{(t)}\rangle_A \otimes |0\rangle_D \right)$$

- $U_C^{(t)}$ **(Clifford Frame):** Deterministic geometric coordinate mapping.
- $\tilde{P}^{(t)}$ **(Virtual Pauli Frame):** Tracks discrete stochastic errors via virtual X and Z vectors: $\tilde{P}^{(t)} = \bigotimes_{j=0}^{n-1} X_j^{x_j^{(t)}} Z_j^{z_j^{(t)}}$. Evaluated via $\mathcal{O}(1)$ bitwise XORs.
- $|\phi^{(t)}\rangle_A$ **(Active Statevector):** A dense $2^{\mathbf{k}}$ array tracking non-Clifford superposition on k *Active* virtual qubits.
- $|0\rangle_D$ **(Dormant Qubits):** The $n-k$ qubits mathematically locked in $|0\rangle$.
- $\gamma^{(t)}$ **(Global Scalar):** Tracks continuous phases and deferred normalization.

What happens?

$$|\psi^{(t)}\rangle = \gamma^{(t)} U_C^{(t)} \tilde{P}^{(t)} \left(|\phi^{(t)}\rangle_A \otimes |0\rangle_D \right)$$

If a gate in the circuit is a Clifford C, we simply absorb it – $U_C^{(t+1)} = C \cdot U_C^{(t)}$.

If a gate is an "active" operation O, its physical Pauli maps to a virtual Pauli via clifford rules. Thanks stim tableau!

$$PU_C^{(t)} = U_C^{(t)} \tilde{P} \quad \text{(Pauli Mapping)}$$

Virtual Compression (The "Secret Sauce")

$\tilde{P}$ can touch many virtual qubits (e.g., $\tilde{P} = X_0 \otimes Z_1$). How do we apply it without evaluating a full 2^n statevector?

Lemma 1: Exact Pauli Compression

The Multi-Level compiler constructively synthesizes a sequence of virtual Cliffords V to compress *any* multi-qubit Pauli onto a **single virtual axis** v. ($V\tilde{P}V^\dagger = P_v$).

Lemma 2: Zero-Cost Dormancy

If V uses a Dormant qubit as a control (e.g., $\text{CNOT}_{D\to A}$), it mathematically acts as the Identity on the Active statevector. **Zero FLOPs required.**

Applying the Compression: Where does V go?

If we apply virtual Cliffords V to compress an operator $O(\tilde{P})$, how is the global state preserved?

The compiler mathematically inserts the identity $V^\dagger V = I$ into the factored state, effectively **pushing it forward**:

$$\dots \underbrace{U_C^{(t)} V^\dagger}_{\text{1. Offline Frame}} \underbrace{\left[V O(\tilde{P}) V^\dagger \right]}_{\text{2. Localized Op}} \underbrace{\left(V \tilde{P}^{(t)} V^\dagger \right)}_{\text{3. Pauli Tracker}} \underbrace{V \left(|\phi^{(t)}\rangle_A \otimes |0\rangle_D \right)}_{\text{4. Active Array}}$$

The transformation V distributes exactly where it is needed:

1. **Clifford Frame:** Absorbs the inverse offline ($U_C \leftarrow U_C V^\dagger$).
2. **The Operation:** Compresses to act on a single virtual axis ($O(P_v)$).
3. **Pauli Frame:** Is dynamically conjugated via $\mathcal{O}(1)$ bitwise XORs.
4. **Active Array:** Undergoes the transformation V (which costs **zero FLOPs** if controlled by a dormant qubit!).

Walkthrough: A Toy Example

Walkthrough: Step-by-Step Factored State

Offline Compiler Online VM

Circuit (2 Qubits)

$\rightarrow$ Init $|00\rangle$

Phys $H(q_0)$

Phys $T(q_0)$

Phys $\mathrm{CX}(q_0 \rightarrow q_1)$

q_1

System State at t

$U_C = I, \quad \tilde{P}^{(t)} = I$

$A = \emptyset, \quad D = \{0, 1\} \quad (\mathbf{k} = \mathbf{0})$

$|\phi\rangle_A = [1], \quad \gamma = 1.0$

Walkthrough: Step-by-Step Factored State

Offline Compiler Online VM

Circuit (2 Qubits)

Init $|00\rangle$

$\rightarrow$ Phys $H(q_0)$

Phys $T(q_0)$

Phys $\mathrm{CX}(q_0 \rightarrow q_1)$

Measure q_1

System State at t

Action: Update deterministic frame offline.

$U_C = H_0$

$\tilde{P}^{(t)} = I$

$A = \emptyset, \quad D = \{0, 1\} \quad (\mathbf{k} = \mathbf{0})$

$|\phi\rangle_A = [1]$ *(Zero FLOPs!)*

Walkthrough: Step-by-Step Factored State

Offline Compiler Online VM

Circuit (2 Qubits)

Init $|00\rangle$

Phys $H(q_0)$

$\rightarrow$ Phys $T(q_0)$ *(Compile)*

Phys $\mathrm{CX}(q_0 \rightarrow q_1)$

Measure q_1

System State at t

Compile T-Gate (Rotation about Z) (offline):

$T \propto \exp\left(-i\frac{\pi}{8}\mathbf{Z_0}\right)$. We track the generator $\mathbf{Z_0}$!

Compiler applies $V = H_0$ to align $X_0 \rightarrow Z_0$ to make this diagonal phase.

Walkthrough: Step-by-Step Factored State

Offline Compiler Online VM

Circuit (2 Qubits)

Init $|00\rangle$

Phys $H(q_0)$

$\rightarrow$ Phys $T(q_0)$ *(Execute)*

Phys $\mathrm{CX}(q_0 \rightarrow q_1)$

Measure q_1

System State at t

Execute T-Gate (Dimensionality Expansion) (online):

$U_C \leftarrow U_C \cdot H_0 = I$ *(Virtual $V^\dagger$ pushes into U_C)*

Axis q_0 activates.

$A = \{0\}, \quad D = \{1\} \quad (\mathbf{k = 1})$

$|\phi\rangle_A = [1, e^{i\pi/4}]^T$ *(Array Doubles!)*

$\gamma = e^{i\pi/8}/\sqrt{2}$ *(Absorbs global phase)*

Walkthrough: Step-by-Step Factored State

Offline Compiler Online VM

Circuit (2 Qubits)

Init $|00\rangle$

Phys $H(q_0)$

Phys $T(q_0)$

$\rightarrow$ Phys $\mathrm{CX}(q_0 \rightarrow q_1)$

Measure q_1

Key Insight:

The physical CX entangles the qubits, but because it is a Clifford, the Active Array doesn't care! It costs zero FLOPs.

System State at t

Physical Entanglement (offline):

Clifford gate maps directly into the offline frame.

$U_C \leftarrow \mathrm{CX}_{0\rightarrow 1} \cdot U_C = \mathrm{CX}_{0\rightarrow 1}$

$A = \{0\}, \quad D = \{1\} \quad (\mathbf{k} = \mathbf{1})$

$|\phi\rangle_A = [1, e^{i\pi/4}]^T$

The array is completely untouched!

Walkthrough: Step-by-Step Factored State

Offline Compiler Online VM

Circuit (2 Qubits)

Init $|00\rangle$

Phys $H(q_0)$

Phys $T(q_0)$

Phys $\mathrm{CX}(q_0 \to q_1)$

$\to$ Measure q_1

System State at t

Action: Projective Measurement (online).

Outcome m.

Physical Z_1 pushes through $\mathrm{CX}_{0\to1}$ to $\mathbf{Z_0} \otimes \mathbf{Z_1}$.

Virtual $V = \mathrm{CX}_{1\to0}$ compresses it to Z_0. Measure Active q_0.

$\tilde{P}^{(t)} \leftarrow$ Updates $\mathbf{x}, \mathbf{z}$ tracking.

$A = \emptyset, \quad D = \{0, 1\} \quad (\mathbf{k} = \mathbf{0})$

$|\phi\rangle_A = [1] \quad$ *(Array Halves!)*

$\gamma \leftarrow \gamma / \sqrt{p_m}$

Software Architecture

The "Aha!" Moment: Theorem 1

Look closely at the walkthrough. When did the Clifford Frame (U_C) and active bounds (A, D) update? When did the statevector ($|\phi\rangle_A$) and Pauli tracker ($\tilde{P}$) compute?

Theorem 1: Clifford Frame Determinism

The evolution of the Clifford Frame U_C and the dimensional bounds of the active set ($k_{\max}$) depend *exclusively* on the sequence of physical operations. They are strictly independent of probabilistic measurement outcomes or continuous non-Clifford amplitudes.

Consequence: We do not need to do heavy tableau math online. We cleanly split the simulation into an **Offline Multi-Level Compiler** and an **Online Virtual Machine**.

Decoupling Computational Complexity

1. **Multi-Level Compiler (Offline, once per circuit):**
   - Requires the $\mathcal{O}(Gn^2)$ topological matrix tracking.
   - Evaluates optimal Pauli Compressions to single virtual axes.
   - Computes global algebraic optimizations over the timeless **Heisenberg IR (HIR)**.

2. **The Schrödinger Virtual Machine (Online, once per shot):**
   - Clifford is just bit operations
   - Non-clifford bounded $\mathcal{O}(2^{k_{\max}})$.
   - No dynamic memory allocation, no tableau tracking. Executes purely localized RISC bytecode.
   - *Note: C++ engineering making the VM fast (32-byte opcodes, branchless hardware popcounts) is a topic for a future deep-dive!*

Mapping the Theory to the Codebase

1. Front-End: Trace & Map to Virtual Basis

$$\underbrace{O(P_{\text{lab}}) \xrightarrow{U_C^{(t)}} O(\tilde{P})}_{\text{Heisenberg Mapping}}$$

Tracks physical coordinate frame → **Offline Tracker**
`sim.inv_state` (U_C)

Stores multi-qubit generator $O(\tilde{P})$ → **Heisenberg IR (HIR)**
`T_GATE +X0*Z1`

2. Back-End: Lowering & Virtual Compression

synthesize V →

Tracks accumulated virtual gates → **Offline Tracker**
`ctx.v_cum` (V)

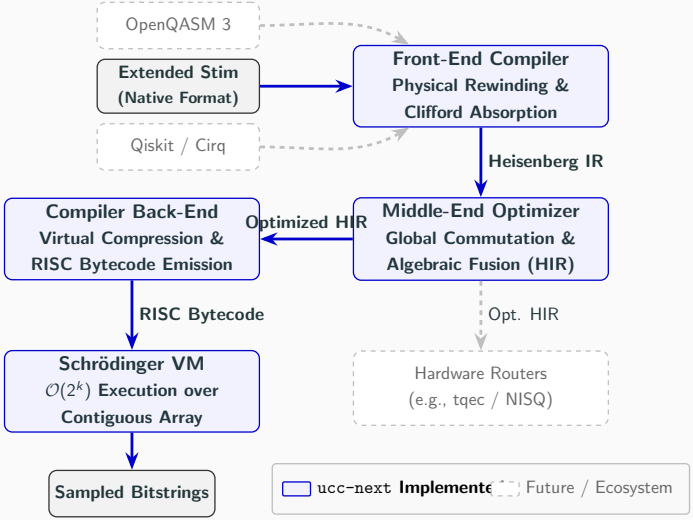
$$\underbrace{V\,O(\tilde{P})\,V^\dagger}_{\text{Localized Math}} \qquad \underbrace{V\left(|\phi\rangle_A \otimes |0\rangle_D\right)}_{\text{Array Routing}} \qquad \underbrace{V\,\tilde{P}^{(t)}\,V^\dagger}_{\text{Pauli Tracker}}$$

Emits $O(P_v)$ → **VM Math Op**
`OP_PHASE_T 1`

Emits array V → **VM Routing & Frame Ops**
`OP_ARRAY_CNOT 0,1`

Emits frame V → **VM Routing & Frame Ops**

UCC Architecture Diagram



Live Demo

Manually driving compiler and evaluating active bounds

Toy Example

MSC $d = 3$ Cultivation Step

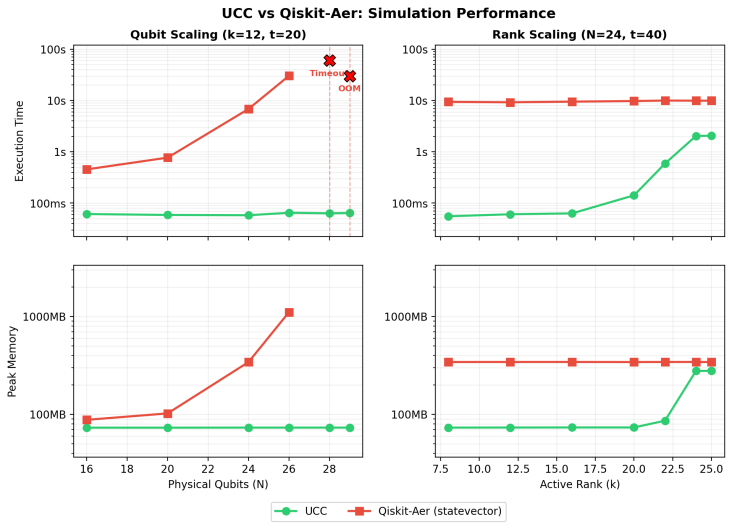
Benchmarks & Results

The “Three Walls” Benchmark

Each circuit has N physical qubits, k peak active non-Clifford virtual qubits, and t total T-gates. We isolate each axis on identical random Clifford+T circuits, comparing UCC against Qiskit-Aer (statevector) on a single CPU core:

- **Panel A — Physical Qubit Wall (N):** Fix k=12, t=20, sweep N.
- **Panel B — Active Rank Wall (k):** Fix N=24, t=40, sweep k.

Three Walls: UCC vs. Qiskit-Aer



Dashed/ ✕ = timeout (120 s) / OOM (6.5 GB). Single-threaded, Intel Xeon 8259CL, 7.2 GB RAM.

Next - Scaling Up: Magic State Cultivation

Goal: Evaluate Magic State Cultivation (MSC) protocols at scale.

Phase 1 Validated-ish ($d = 5$ Cultivation Step):

- The SOFT simulator (Li et al. 2025) previously required a 16-GPU cluster to process online dynamic tableaus for this 42-qubit circuit.
- Validated UCC matches output in high-noise, estimate full comparison is $45/29H (to match) -$430/285H (to do at MSC paper confidence levels) on AWS.

Next Target: Estimating the runtime and cost required to execute the *full* 463-qubit ($d = 7$) end-to-end cultivation protocol.

Next Steps & Discussion

Feedback & Discussion

1. **Project Scope:** Is this `ucc-next`? Should it replace UCC or be a new thing?
2. **Publication Strategy:** What are the most critical benchmarks to include an the initial paper?
3. **Narrative Framing:** Does positioning UCC as "A Multi-Level Compiler & Execution Engine for the Early Fault Tolerance Era" resonate?

Thank You!

Questions?